

Revision Notes for Class 12 Computer Science Chapter 3 Stack

In computer science, a data structure organises and manages collections of data. Stacks are a type of linear data structure, different from other structures like arrays, linked lists, or queues. In a stack, items are added and removed from only one end, called the top, following the Last-In-First-Out (LIFO) principle.

This means the element added last will be removed first, similar to how we manage piles of plates or books. The fundamental stack operations are PUSH, to insert an element at the top, and POP, to remove the top element. If the stack is full, a PUSH operation may result in a condition called “overflow”; if empty, a POP leads to “underflow.”

Key Characteristics of Stacks

Stack elements are organised in sequence—there is no way to add or remove elements except at the top. Stacks can be visualised by daily life analogies:

- Piles of books or plates—insert and remove from the top only.
- Vertical piles of chairs.
- Bangles worn one after another on the wrist.
- Boxes stacked in a kitchen shelf or pantry.

Stacks are not just theoretical; they solve many real-world computing problems. For example, exam systems, web browsers, and programming tools use stacks to keep track of data and operations.

Applications of Stack in Programming

In daily programming, stacks play a vital role. Some of the most important applications include:

- **Reverse a string:** The latest character entered becomes the first one out, making string reversal simple using stacks.
- **Redo/Undo in editors:** Each edited action is stored in a stack, helping with undo and redo commands.
- **Web browsing history:** Browsers keep track of visited pages using stacks, making BACK and FORWARD buttons possible.
- **Parentheses matching:** Compilers and interpreters check for proper nesting of parentheses using stacks.

Stack Operations: PUSH and POP

There are two primary stack operations:

- **PUSH:** Adds a new element to the top. If the stack is full, trying to add one more gives an overflow error.
- **POP:** Removes the element at the top. Removing from an empty stack produces an underflow error.

In Python, stacks are easily implemented with the built-in list type. The `append()` method is used to PUSH (insert) an item, and `pop()` removes (POPs) it from the top.

Implementing Stack in Python

Stack functions can be custom-defined for clarity:

- `isEmpty(stack)` – Checks if the stack is empty.
- `opPush(stack, element)` – Pops in a new element.
- `opPop(stack)` – Removes and returns the top element.
- `top(stack)` – Shows the current top element without removing it.

- `display(stack)` – Displays all elements from top to bottom.

A sample Python program will create a stack, allow items to be pushed and popped, show the top, and eventually empty the stack. Trying to pop from an empty stack gives an underflow message; pushing beyond maximum size would overflow in fixed-size implementations (though Python lists grow dynamically).

Notations for Arithmetic Expressions

Arithmetic expressions can be written in three main notations:

Infix: Operators are written between operands (e.g., $x + y$).

Prefix (Polish): Operators are written before operands (e.g., $+xy$).

Postfix (Reverse Polish): Operators are written after operands (e.g., $xy+$).

Type of Expression	Description	Example
Infix	Operators are placed between operands	$x * y + z$ $3 * (4 + 5)$ $(x + y) / (z * 5)$
Prefix (Polish)	Operators are before operands	$+z*xy$ $*3+45$ $/+xy*z5$
Postfix (Reverse Polish)	Operators are after operands	$xy*z+$ $345+*$ $xy+z5*/$

Infix is commonly used in mathematics and programming, but computers work more efficiently using prefix or postfix, as no parentheses are required if the order of operators is maintained.

Converting Infix to Postfix Notation

The conversion of infix to postfix is a step-by-step procedure involving stacks. Each operator, operand, and parenthesis is processed in order:

1. Create an empty string (postExp) and process each character of the infix expression.
2. Push left parenthesis onto the stack.
3. If a right parenthesis is found, pop operators from the top and append to result until you reach a left parenthesis (discard both parentheses).
4. If an operator has lower precedence than the one at the top of stack, pop and append from stack to result until the correct rank is found, then push new operator.
5. Operands are directly appended to the result string.
6. At the end, pop remaining operators from the stack into the result string.

This algorithm helps computers and calculators evaluate expressions quickly and correctly. Parentheses and operator precedence are handled easily.

Evaluating Postfix Expressions

Stack-based evaluation of postfix expressions is simple. Read left to right:

1. For each symbol: If it's an operand, push to the stack.
2. If it's an operator, pop two operands and apply the operator.
3. Push the result onto the stack.
4. The final answer will be at the top of the stack after processing all characters.

If more than one number remains in the stack at the end, the postfix expression was invalid. This method ensures calculations proceed in the correct order, even for complex formulas.

Summary Points

- A stack allows insertion and deletion from only one end, always the top.
- It follows the LIFO rule—Last In, First Out.
- Basic operations are PUSH (insert) and POP (delete).
- Python lists can be used as stacks, using `append()` and `pop()` functions.
- Arithmetic expressions can be written as infix, prefix, or postfix forms; computers easily evaluate prefix and postfix using stacks.
- Stack data structure helps in conversion and evaluation of algebraic expressions in programming.